SoK: Authorization for Autonomous Agents

Anonymous Author(s)

ABSTRACT

An autonomous agent calls tools and Model Context Protocol (MCP) servers while carrying a bearer token (an OAuth 2.0 access token or an MCP authorization token) on each request. Because the agent is driven by untrusted input, its authorization layer is part of the attack surface: a token not bound to the server it is sent to can be replayed or passed through, an over-broad scope hands an injected agent far more than any task needs, and an on-behalf-of token with no verifiable delegation chain leaves "who authorized this?" unanswerable. A fast-growing research literature studies agent security, but it concentrates on model-level attacks (prompt injection, tool poisoning) and proposes individual mechanisms; it does not systematize the authorization layer. We survey 44 research works and show that two authorization properties, audience binding and confused-deputy / dynamic client registration, are treated as mere specification obligations rather than as properties anyone tests or enforces. We then systematize agent authorization along three axes no prior work combines: (i) by *pre-deployment testability*, meaning which threats are decidable from a token, a key set, and a declared tool contract before the agent runs; (ii) by *normative provenance*, a threat-to-clause mapping to the OAuth RFCs, the MCP authorization specification, OWASP, and CWE; and (iii) by the *static-versus-runtime enforcement line*. Across fifteen threats in six categories, we find six are statically contract-verifiable and nine are irreducibly runtime, and we argue this line should guide both tooling and standardization.

1 INTRODUCTION

When an agent calls a tool or an MCP server on a user's behalf, a bearer token travels with the request. The failure modes are OAuth failure modes, but with sharper consequences, because the caller is autonomous and can be steered by untrusted input such as a retrieved document or a tool description [9]. A token not bound to the resource it is presented to can be replayed against, or forwarded through, a different server (a confused deputy); an on-behalf-of (OBO) token with no verifiable delegation chain makes accountability impossible; an omnibus scope such as `admin:*` hands an injected agent the keys to everything; and a non-expiring credential never stops being useful to whoever obtains it.

A large and fast-moving body of research now studies agent and MCP security. As our survey (§5) shows, it is dense on *model-level* attacks (prompt-injection-driven escalation and tool poisoning) and it proposes many *individual* defense mechanisms (privilege controllers, delegation protocols, identity architectures). What it does not do is systematize the *authorization* layer itself: no surveyed work sorts agent-authorization threats by whether a defender can test them before deployment, maps each threat to the specific normative clause it violates, or names the line between what can be enforced statically at the trust boundary and what can only be enforced at runtime. The two authorization properties that matter most for token safety, audience binding and the confused-deputy / dynamic-client-registration pair, are treated across 44 works as specification obligations to cite rather than as properties to verify (§5).

Contributions. (i) We give a source-grounded threat model of fifteen agent-authorization threats in six categories, each with its broken invariant, the conditions under which it is exploitable, a mitigation, and its normative source (§2, §3). (ii) We systematize this space along three axes no prior work combines: testability, normative provenance, and the static/runtime enforcement line (§4). (iii) We survey 44 research works on agent authorization and make the coverage gap precise: audience binding and confused-deputy are the under-formalized seam (§5). (iv) We argue which authorization properties a standard or a pre-deployment tool could mandate, and release an open-source analyzer for the statically decidable subset (§4).

2 BACKGROUND

Tokens and claims. An OAuth 2.0 access token authorizes a client to call a resource on a user's behalf; in agent deployments it is usually a JSON Web Token (JWT), a signed set of claims. The claims that govern authorization are the issuer (`iss`), the intended audience (`aud`, the resource the token is *for*), the granted `scope`, the expiry (`exp`), and, for delegation, the authorized party (`azp`, naming the client) and the actor (`act`, naming who is acting). A verifier must check the signature against the issuer's keys, that `aud` names itself, that the algorithm is on an allow-list, and that required claims hold (RFC 8725). RFC 9068 profiles JWT access tokens and makes audience and issuer validation mandatory for that profile; RFC 8707 (resource indicators) lets a client request a token bound to a specific resource.

Delegation and the `act`/`azp` claims. OAuth token exchange (RFC 8693) issues a delegated token carrying an `act` claim that names the acting party and a `may_act` claim stating who is *permitted* to act for the subject. A single `act` claim records one hop; a chain (user → agent → sub-agent) requires *nested* `act` claims, and transitive delegation is only verifiable if each exchange enforces `may_act` and the verifier walks the full nested chain. `azp` names the client that obtained the token but does not by itself establish a delegation chain: it identifies the immediate party, not the sequence of authorizations behind it. This is why `azp`/`act` alone are necessary but not sufficient: without enforced `may_act` and nested-chain validation, "who authorized this?" remains unanswerable across more than one hop.

MCP, identity, and the confused deputy. The Model Context Protocol standardizes how an agent discovers and calls tools; its authorization specification inherits OAuth and *prohibits* passing a client's token through to an upstream API. Dynamic client registration (DCR, RFC 7591) lets clients register at runtime, which an MCP proxy may expose. Workload-identity substrates (WIMSE;

Conference'17, July 2017, Washington, DC, USA
2026. ACM ISBN 978-x-xxxx-xxxx-x/YY/MM…$15.00
https://doi.org/10.1145/nnnnnnn.nnnnnnn

SPIFFE/SPIRE) would give each agent instance a verifiable, short-lived identity distinct from the human's, but are rarely deployed. The confused-deputy problem (CWE-441), where a more privileged party misuses its authority on behalf of a less privileged one, is the classical frame for token passthrough and consent-flow abuse.

The agent trust model. Two properties of the agent setting sharpen every OAuth failure above. First, the caller is *autonomous*: it chooses which tool to call and with which arguments, so a credential it holds is exercised without a human reviewing each use. Second, its inputs are *untrusted*: an agent reads documents, web pages, and tool descriptions an attacker may control, so prompt injection can steer it into requests the user never intended [9]. Together these mean the agent must be treated as a party that can be induced to misuse whatever its token permits, the assumption our threat model adopts and that recent multi-agent security agendas make explicit [32]. The consequence for authorization is concrete: the token an agent carries, not a human's judgement at call time, is the last line of defense, so the properties that token enforces are what bound the blast radius of an injected agent. This is why we treat the token-level, statically checkable properties as first-class rather than as spec footnotes.

3 A THREAT MODEL FOR AGENT AUTHORIZATION

We organize the space into six categories. Table 1 lists fifteen threats with the invariant that must hold, a one-line mitigation, and whether a single-token or server-manifest check can decide the threat before deployment. We then give, per category, the exploit condition and the defense.

Audience binding (MCP-01/02). A bearer token names, in its `aud` claim, the resource it is for. The exploit has two shapes. In *passthrough* (MCP-01), an MCP server that receives a client's token forwards it unchanged to an upstream API; because the upstream often shares an issuer it accepts the token, and the MCP server has acted as a confused deputy wielding the full authority of the user's token. In *replay* (MCP-02), a token minted with no resource indicator, or one whose `aud` a verifier never checks, is accepted at any server the issuer serves, so an attacker who captures one token reaches many. Both are properties of claims the token carries and of the server's declared identity, so a defender mints the adversarial token and confirms rejection before deployment. The defense is to request resource-bound tokens (RFC 8707), to verify `aud` names this server, and to honor the MCP prohibition on passthrough. Among the works we survey only Agentic JWT [8] and one enterprise framework [23] treat audience binding as a property to enforce rather than a specification clause to cite (§5).

Delegation (DEL-01..04). When an agent acts for a user, an on-behalf-of (OBO) token should carry an `act` claim naming the acting party (RFC 8693). A single `act` records one hop; a chain user → agent → sub-agent needs *nested* `act` claims, and the chain is trustworthy only if every exchange along it enforced the `may_act` claim that states who may act for the subject. The exploit: with a missing or forged `act` (DEL-01) an action cannot be attributed, and with an unverified chain (DEL-02) a sub-agent silently exceeds what was delegated, a transitive-trust escalation. Two of the four are statically checkable: a verifier confirms an `act` claim is present and the nested chain is well-formed from the token alone. The other two are runtime: whether the token endpoint *enforced* `may_act` on each exchange (DEL-03), and whether a delegated rather than an impersonated token was issued (DEL-04), are properties of the issuing flow, not of the presented token. Delegation protocols that bind each action to user intent and bound a sub-agent's authority target this category [8, 22, 27, 39].

Scope (SCOPE-01..03). The `scope` and `exp` a token carries bound what an injected agent can do with it. The exploit: an omnibus scope such as `admin:*` (SCOPE-01) hands an injected agent far more than any task needs, and a non-expiring token (SCOPE-02) stays useful to whoever later obtains it. Both are read directly from the token, so a check against a declared allow-list and a maximum lifetime is a pre-deployment test. The third, an agent reusing the human's root credential (SCOPE-03), is a property of how the deployment provisions credentials, not of any single token, and so is runtime. Least-privilege frameworks enforce the scope constraint at runtime with monotonic confinement [36], and measurements show over-privilege is common in practice [17]; our point is that the carried scope and lifetime are checkable before the agent ever runs.

Agent identity (AID-01/02). An agent needs a verifiable identity distinct from the human's so its actions can be attributed and its access revoked. The exploit: treating a session token as authentication (AID-01), or giving the agent no workload identity at all (AID-02), leaves no principal to revoke or audit. The defense is a deployed workload-identity substrate (WIMSE, SPIFFE/SPIRE) that issues each agent instance a short-lived, verifiable credential [24, 33, 38]. Both are runtime: with no identity layer deployed there is nothing in a presented token to check against, which is why they sit on the runtime side of the line.

Confused deputy (CD-01/02). An OAuth proxy sits between many clients and a resource. The exploit: if the proxy reuses one consent across clients, or matches `redirect_uri` loosely, one client inherits another's authorization (CD-01); and if it exposes dynamic client registration without vetting (RFC 7591), an attacker registers a client the proxy then trusts (CD-02). These are properties of a live consent flow and of how the proxy matches redirects at request time, so they cannot be decided from a token or a static manifest. Protocol-level analyses of MCP surface exactly these trust-propagation gaps [16, 19], yet across the literature the confused deputy appears almost only as a named taxonomy entry, not an enforced property (§5).

Agency (AGT-01/02). The agency threats are where authorization meets the model. The exploit: untrusted input drives the agent to escalate privilege (AGT-01) or to call a tool whose description has been poisoned (AGT-02), so the agent itself issues the malicious request with its own legitimate token. No token check helps, because the token is valid and the request is the agent's own; the decision depends on the executing context. The defenses are runtime: per-action authorization or a human in the loop so an injection cannot escalate on its own [36], and vetting of tool descriptions before they reach the model [29, 44]. This category is the most densely studied in the literature (§5), which is part of our point: the field has concentrated on the model-level attacks and under-formalized the token-level guarantees.

Table 1: Agent-authorization threats by category, with the invariant that must hold, a mitigation, the normative source, and whether the threat is *statically* decidable from a single token or server manifest before deployment (√) or is irreducibly runtime (×). Sev.: severity.

ID	Sev.	Broken invariant (exploit)	Mitigation	Source	Static
Audience binding					
MCP-01	crit	token accepted with wrong/any audience (passthrough, replay)	verify `aud` names this resource; reject forwarded tokens	RFC 8707/9728; MCP [1, 2, 13]	√
MCP-02	high	token carries no resource indicator (unbound audience)	request resource-bound tokens; require `aud`	RFC 8707 [2]	√
Delegation					
DEL-01	high	missing or forged `act` claim (no accountable actor)	require a signed `act` claim on OBO tokens	RFC 8693 [14]	√
DEL-02	high	unverifiable nested delegation chain (transitive-trust abuse)	validate the full nested `act` chain	RFC 8693 [14]	√
DEL-03	high	`may_act` not enforced on exchange	enforce `may_act` at the token endpoint	RFC 8693 [14]	×
DEL-04	med	impersonation where delegation was required	issue delegated (not impersonated) tokens	RFC 8693 [14]	×
Scope					
SCOPE-01	high	over-scoped agent credential (`admin:*`)	least-privilege scopes per task	RFC 9700; OWASP [18, 26]	√
SCOPE-02	high	non-expiring / long-lived token	short `exp`; rotation	RFC 8725 [34]	√
SCOPE-03	high	agent reuses the human's root credential	distinct agent credential	OWASP [26]	×
Agent identity					
AID-01	high	session used as authentication	bind to a verifiable workload identity	WIMSE/SPIFFE [11, 37]	×
AID-02	med	no distinct workload/agent identity	issue per-agent SVID/credential	WIMSE/SPIFFE [11, 37]	×
Confused deputy					
CD-01	crit	proxy acts on a client's behalf without per-client consent	per-client consent; exact `redirect_uri` match	CWE-441; MCP [1, 21]	×
CD-02	high	dynamic client registration abuse	vet/limit DCR; bind registrations	RFC 7591 [30]	×
Agency					
AGT-01	crit	prompt-injection-driven privilege escalation	per-action authorization; human-in-loop	OWASP [25, 26]	×
AGT-02	high	tool poisoning / rug-pull via tool metadata	vet tool descriptions; pin versions	OWASP [26]	×

A worked scenario. Consider a travel-planning agent that books flights through an MCP server, which in turn calls an airline API. The user authenticates once; the agent then receives an access token and acts autonomously. Three failures from Table 1 compose. If the booking server forwards the user's token to the airline API rather than exchanging it (MCP-01), and the airline shares the issuer, a malicious or compromised airline endpoint replays the user's token against any resource that issuer serves. If the token carries scope `travel:*` rather than `flights:book` (SCOPE-01), a prompt injected through a hotel-review page the agent reads can drive it to cancel bookings or read the user's itinerary history, because the token already permits it. And if the delegated token carries no `act` claim (DEL-01), neither the airline nor an auditor can later establish that the agent, rather than the user directly, issued the request. Each of these three is visible in the token and the server's declared contract *before the agent runs*: a defender mints a token with the wrong audience, an over-broad scope, and a missing `act`, and confirms the verifier rejects each. The consent flow that let the airline register as a client in the first place (CD-02) and the prompt injection that triggered the misuse (AGT-01) are not visible this way; they need a live consent flow and a running agent. The same scenario therefore straddles the line, and the line tells the team which half to gate in CI and which half needs a runtime control.

4 SYSTEMATIZING THE SPACE

Our systematization rests on three axes that, together, no prior work applies to agent authorization (§5).

(1) Pre-deployment testability. A threat is *statically decidable* when a defender can reproduce it from a single token, the issuer's key set, and the server's declared tool contract, *without running the agent*. By this criterion six of the fifteen threats are statically decidable: audience binding (MCP-01/02), the `act`-chain delegation checks (DEL-01/02), and scope and expiry (SCOPE-01/02). Each is a property of claims a token carries or of a server's declared identity, so a defender can mint the adversarial token an attacker would and confirm the verifier rejects it, as a check that runs on every build. The remaining nine depend on a live consent flow (CD-01/02), server-side enforcement or a human-in-the-loop (DEL-03/04, SCOPE-03, AGT-01/02), or a deployed workload-identity substrate (AID-01/02), and cannot be settled from a token.

(2) Normative provenance. Each threat maps to the clause it violates: audience binding to RFC 8707 and the MCP passthrough prohibition; delegation to RFC 8693; scope and expiry to RFC 9700 and RFC 8725; registration to RFC 7591; confused deputy to CWE-441; agency to the OWASP LLM and Agentic Top-10s [25, 26]. This lets a practitioner trace a defense to the authority that mandates it, a mapping the surveyed literature does not provide.

(3) The static/runtime enforcement line. The defenses in the literature scatter across a line they never name: *static* enforcement at the trust boundary (control/data-flow extraction [4], capability

attestation, contract checks on tokens and key sets) versus *runtime* enforcement during execution (monotonic privilege confinement [36], pre-action authorization [40], mandatory access control [12], out-of-band policy [20]). Drawing the line explicitly reframes several "runtime-only" defenses as compensating controls for threats a pre-deployment contract check could have excluded: if audience binding and scope are verified statically, a runtime controller has strictly less to contain. Figure 1 places the fifteen threats on this line.

An instrument for the static subset. We release an open-source analyzer that decodes a presented token and reports cited findings for the six statically decidable threats (audience binding, `act`-chain delegation, scope, and expiry) against a policy. It is stateless, makes no network calls, and runs in CI or beside the agent, so a team red-teams its own authorization before shipping. The taxonomy names the other nine as out of static scope, which is itself the result: it says what a pre-deployment tool, or a tightened standard, can and cannot be expected to guarantee.

5 RELATED WORK

We surveyed 44 research works on authorization, delegation, identity, and token security for agents and MCP (2023–2026); all arXiv identifiers were verified to exist. They divide into *threat taxonomies* and *point mechanisms*, and neither systematizes authorization by our three axes.

Taxonomies and surveys. MCP security surveys enumerate attack surfaces across the server lifecycle [10], and recent systematizations consolidate attacks and defenses for the ecosystem [7] and for multi-agent systems end to end [43]. Threat taxonomies name authorization-adjacent risks explicitly, including the confused deputy and over-broad consent [35], and a defense-placement taxonomy classifies where in the architecture a control sits [31]. Protocol-level analyses dissect the MCP specification itself, finding missing capability attestation and implicit trust propagation across chained servers [6, 19]. These organize by attacker type, lifecycle phase, defense placement, or ecosystem component, and none sorts threats by pre-deployment testability or binds each to the clause it violates.

Delegation and identity mechanisms. A line of work makes delegation verifiable: Agentic JWT binds each agent action to a user intent in the token [8]; AIP carries delegation chains across MCP and A2A boundaries [27]; bounded-delegation and stateful-effect schemes keep a sub-agent within what was delegated [22, 39]; and secret-use delegation and residual authorization treat scoped secret handling and revocation [3, 45]. Identity work gives agents a verifiable, distinct principal through WIMSE and SPIFFE/SPIRE [24, 28, 33], and full governance architectures combine registration, identity, and access tokens [38]. These are the mechanisms our delegation and identity categories draw on, each addressing a slice rather than the authorization layer as a whole.

Privilege control and model-level defenses. Runtime privilege frameworks enforce least privilege with symbolic rules and monotonic confinement [36], move the authorization decision before each tool call [40], apply mandatory access control to agent tool use [12], isolate tools in separate environments [42], and enforce policy out-of-band at a trusted boundary [20]; measurements show over-privilege is widespread in practice [17]. For the model-level threats, by-design defenses extract control and data flow so untrusted input cannot alter program flow [4], benchmarks quantify injection success [5], and tool-poisoning work measures and defends the description surface [29, 41, 44, 46], with indirect prompt injection the seminal threat [9]. Each is one point on the static/runtime line; none draws the line, and the densest coverage is on the model-level attacks rather than the token-level guarantees.

The coverage gap. Scoring every work on seven threat categories (Table 2) makes the gap visible: two columns are thin. Audience binding is a central contribution in essentially one research work (Agentic JWT [8]) plus one enterprise framework [23], and confused-deputy / registration appears only as a named taxonomy entry, almost never as a verified or enforced property. Prompt injection and tool poisoning, by contrast, are densely covered. The field has thoroughly characterized the model-level attacks and under-formalized the token- and consent-level authorization guarantees, the seam this SoK occupies.

6 DISCUSSION: WHAT A STANDARD COULD MANDATE

The testability axis has a direct standardization reading. The six statically decidable threats are exactly the ones a profile could make mandatory-to-verify and a conformance test could check: an MCP or OAuth profile that *requires* resource indicators (RFC 8707), rejects passthrough, mandates nested `act` validation, and caps scope and lifetime would move these threats from "deployer's responsibility, usually missed" to "checked at the boundary." Table 3 makes this concrete: for each statically decidable threat it states the clause a profile would cite, the requirement it would impose, and the conformance test that would decide it. Every test is a token minted to violate the requirement together with the expected rejection, so the profile ships with its own executable check rather than prose a deployer interprets. This is the sense in which the static subset is not only analyzable but *standardizable*: a requirement a conformance suite can adjudicate from inputs the deployer already holds.

A research agenda for the runtime nine. The nine runtime threats cannot be closed by a static profile and instead name where the ecosystem still lacks deployable mechanism. They are not uniformly open: each needs a different substrate, and sorting them this way turns the taxonomy's right-hand column into an agenda. Confused deputy and over-broad registration (CD-01/02) need a consent record that is auditable after the fact, so that which client obtained which grant is reconstructable rather than trusted. Delegation enforcement and impersonation (DEL-03/04) need a runtime that honors a `may_act` constraint and distinguishes delegation from impersonation at the point of use, not only at mint time. Credential reuse (SCOPE-03) needs sender-constrained tokens so a leaked bearer token is not freely replayable. Agent and workload identity (AID-01/02) need a deployed attestation substrate such as SPIFFE/SPIRE [28] so an agent presents a distinct, verifiable principal. And the agency threats (AGT-01/02) need a trusted mediation boundary that survives a prompt-injected plan, which the control-



Figure 1: The static/runtime enforcement line. Six of the fifteen threats are contract-verifiable before deployment; the other nine are irreducibly runtime. The line is the paper's organizing result: it says what a pre-deployment tool, or a tightened standard, can and cannot be expected to guarantee.

Table 2: Coverage of the seven agent-authorization threat categories across representative works. √: a central contribution; ○: partially addressed; blank: not addressed. AudBind, audience binding and passthrough; OBO, on-behalf-of delegation; Scope, scope minimization; ConfDep, confused deputy and dynamic registration; WID, workload/agent identity; PI, prompt-injection escalation; Tool, tool poisoning. Two columns, AudBind and ConfDep, are thin.

Work	AudBind	OBO	Scope	ConfDep	WID	PI	Tool
Taxonomies and surveys							
MCP Landscape [10]	○	○	○	○	○	√	√
SoK MCP [7]	○	○	○	○	○	√	√
MCP-38 [35]	○	○	○	√	○	√	√
Enterprise MCP [23]	√	√	√	√	○	√	√
Breaking the Protocol [19]	○			○	○	√	√
Delegation and identity mechanisms							
Agentic JWT [8]	√	√	√	○	√	○	
AIP [27]	○	√	○	○	√		
Bounded Agents [22]	○	√	√	○	○	○	
AID-Guard [39]	○	√	√	○	○	○	
SAGA [38]	○	√	√	○	√	○	
AI Identity [24]	○	√	○	○	√		
aiAuthZ [15]	√	○	○	○	√		
Privilege / runtime control							
Progent [36]			√			√	○
Pre-Action Authz [40]	○	○	√	○	○	√	
MAC framework [12]		○	√	○		√	○
IsolateGPT [42]			√	○	○	√	○
If Agents Were Angels [20]	○	○	√	○	○	√	○
Model-level attacks and defenses							
Indirect PI [9]						√	○
CaMeL [4]			√	○		√	○
MCPTox [41]			○			√	√
No-Box MCP [46]						√	√

Table 3: A conformance profile for the statically decidable subset. Each row is a requirement a profile could mandate and a test a suite could run from a token, a key set, and a declared contract, with no running agent.

Threat	Clause	Conformance test
MCP-01/02	RFC 8707	Mint a token whose aud names another resource; the verifier must reject it.
DEL-01/02	RFC 8693	Present a delegated token with a missing or malformed act chain; the verifier must reject it.
SCOPE-01	RFC 9700	Present a token whose scope exceeds the tool's declared need; the contract check must flag it.
SCOPE-02	RFC 8725	Present a token past its exp or beyond a capped lifetime; the verifier must reject it.

and data-flow work [4] approaches but does not yet reduce to a standard. Framing these as the complement of the static subset is the practical payoff of the line: it says precisely which problems a tightened token standard retires and which remain genuinely open for mechanism research.

7 THREATS TO VALIDITY

The MCP specification is fast-moving; this systematization is a dated snapshot. The static/runtime line depends on token and manifest availability; a deployment that hides relevant state reduces what any static check can conclude. Our survey covers research indexed by 2026-10; most agent-authorization work is recent and not yet peer-reviewed, which we mark. A prevalence measurement of the statically checkable weaknesses across public MCP deployments is future work, with a hand-validated sample and coordinated disclosure.

ETHICS CONSIDERATIONS

The systematization uses only public specifications, advisories, and CVEs. The accompanying analyzer runs on tokens the operator already holds and sends nothing anywhere; we probe no third-party

agents or servers and use no real user tokens. Any future measurement will characterize population prevalence, not single deployments, and will follow coordinated disclosure for identifiable high-severity findings. No personal data or human subjects are involved.

OPEN SCIENCE

We release the static analyzer and a machine-readable threat taxonomy (each threat with its invariant and normative sources) under an open licence, so the systematization is inspectable and the statically checkable subset is reproducible. A DOI-pinned artifact will accompany the camera-ready version.

8 CONCLUSION

Agent authorization is a high-stakes surface the research literature has studied around rather than through. We systematize it by pre-deployment testability, by normative provenance, and by the static-versus-runtime enforcement line; we show that six of fifteen threats are contract-verifiable before an agent runs and nine are irreducibly runtime; and we argue this line should guide what a pre-deployment tool checks and what a standard makes mandatory to verify.

LLM USAGE STATEMENT

LLMs were used for editorial purposes in this manuscript, and all outputs were inspected by the authors to ensure accuracy and originality. An LLM-assisted process additionally supported the literature survey of §5: candidate works were gathered with model assistance and each was then verified by the authors to exist, with titles and author lists confirmed against the arXiv API on 2026-10-06; works that could not be verified were excluded, and one paper that its own authors had formally withdrawn was identified this way and removed. The authors are responsible for the selection of works, the coverage judgments in the systematization, and every claim in the paper. One reproducibility limitation applies: the assisting models are not openly available, so the survey's search recall cannot be reproduced exactly, although every cited work is independently verifiable by its arXiv identifier.

REFERENCES

[1] Anthropic and MCP contributors. 2025. Model Context Protocol: Authorization and Security Best Practices. MCP Specification 2025-06-18. https://modelcontextprotocol.io/specification/2025-06-18/basic/security_best_practices.

[2] B. Campbell, J. Bradley, and M. Jones. 2020. Resource Indicators for OAuth 2.0. RFC 8707, IETF. https://datatracker.ietf.org/doc/html/rfc8707.

[3] Moonwon Choi, Seokho Jeong, Sian Choi, and Seunggeun Lee. 2026. ResidualAuth: What Authorization State Must Language Agents Preserve under Revocable Delegation? arXiv:2609.08062 [cs.CR] arXiv preprint. https://arxiv.org/abs/2609.08062.

[4] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. 2026. Defeating Prompt Injections by Design. In *IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*. CaMeL; also arXiv:2503.18813. https://arxiv.org/abs/2503.18813.

[5] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*. Also arXiv:2406.13352. https://arxiv.org/abs/2406.13352.

[6] Yuval Felendler, Parth A. Gandhi, Idan Habler, Yuval Elovici, and Asaf Shabtai. 2026. From Tool Orchestration to Code Execution: A Study of MCP Design Choices. arXiv:2602.15945 [cs.CR] arXiv preprint. https://arxiv.org/abs/2602.15945.

[7] Shiva Gaire, Srijan Gyawali, Saroj Mishra, Suman Niroula, Dilip Thakur, and Umesh Yadav. 2025. Systematization of Knowledge: Security and Safety in the Model Context Protocol Ecosystem. arXiv:2512.08290 [cs.CR] arXiv preprint. https://arxiv.org/abs/2512.08290.

[8] Abhishek Goswami. 2025. Agentic JWT: A Secure Delegation Protocol for Autonomous AI Agents. arXiv:2509.13597 [cs.CR] arXiv preprint. https://arxiv.org/abs/2509.13597.

[9] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*. Also arXiv:2302.12173. https://arxiv.org/abs/2302.12173.

[10] Xinyi Hou, Yanjie Zhao, Shenao Wang, and Haoyu Wang. 2025. Model Context Protocol (MCP): Landscape, Security Threats, and Future Research Directions. arXiv:2503.23278 [cs.CR] arXiv preprint; widely cited. https://arxiv.org/abs/2503.23278.

[11] IETF WIMSE WG. 2025. Workload Identity Practices (WIMSE). Internet-Draft. https://datatracker.ietf.org/doc/draft-ietf-wimse-workload-identity-practices/.

[12] Zimo Ji, Daoyuan Wu, Wenyuan Jiang, Pingchuan Ma, Zongjie Li, Yudong Gao, Shuai Wang, and Yingjiu Li. 2026. Taming Various Privilege Escalation in LLM-Based Agent Systems: A Mandatory Access Control Framework. arXiv:2601.11893 [cs.CR] arXiv preprint. https://arxiv.org/abs/2601.11893.

[13] M. Jones, P. Hunt, and A. Parecki. 2025. OAuth 2.0 Protected Resource Metadata. RFC 9728, IETF. https://datatracker.ietf.org/doc/html/rfc9728.

[14] M. Jones, A. Nadalin, B. Campbell, J. Bradley, and C. Mortimore. 2020. OAuth 2.0 Token Exchange. RFC 8693, IETF. https://datatracker.ietf.org/doc/html/rfc8693.

[15] Sai Varun Kodathala. 2026. aiAuthZ: Off-Host, Identity-Bound Authorization for AI Agents. arXiv:2607.05518 [cs.CR] Technical report. https://arxiv.org/abs/2607.05518.

[16] Qiaomu Li and Ying Xie. 2025. From Glue-Code to Protocols: A Critical Analysis of A2A and MCP Integration for Scalable Agent Systems. arXiv:2505.03864 [cs.SE] arXiv preprint. https://arxiv.org/abs/2505.03864.

[17] Zhihao Li, Kun Li, Boyang Ma, Minghui Xu, Yue Zhang, and Xiuzhen Cheng. 2025. We Urgently Need Privilege Management in MCP: A Measurement of API Usage in MCP Ecosystems. arXiv:2507.06250 [cs.CR] arXiv preprint. https://arxiv.org/abs/2507.06250.

[18] T. Lodderstedt, J. Bradley, A. Labunets, and D. Fett. 2025. Best Current Practice for OAuth 2.0 Security. RFC 9700, IETF. https://datatracker.ietf.org/doc/html/rfc9700.

[19] Narek Maloyan and Dmitry Namiot. 2026. Breaking the Protocol: Security Analysis of the Model Context Protocol Specification and Prompt Injection Vulnerabilities in Tool-Integrated LLM Agents. arXiv:2601.17549 [cs.CR] arXiv preprint. https://arxiv.org/abs/2601.17549.

[20] Marc Millstone, Tyler Akidau, Johannes Brüderl, and Marat Pekker. 2026. If Agents Were Angels, No Governance Would Be Necessary: Out-of-Band Policy Enforcement at a Trusted Tool Boundary. arXiv:2608.27646 [cs.CR] arXiv preprint. https://arxiv.org/abs/2608.27646.

[21] MITRE. 2024. CWE-441: Unintended Proxy or Intermediary (Confused Deputy). https://cwe.mitre.org/data/definitions/441.html.

[22] Xabier Muruaga. 2026. Bounded Agents: Delegation Security for Multi-Agent AI Systems. arXiv:2608.15888 [cs.CR] arXiv preprint. https://arxiv.org/abs/2608.15888.

[23] Vineeth Sai Narajala and Idan Habler. 2025. Enterprise-Grade Security for the Model Context Protocol (MCP): Frameworks and Mitigation Strategies. arXiv:2504.08623 [cs.CR] arXiv preprint. https://arxiv.org/abs/2504.08623.

[24] Takumi Otsuka, Kentaroh Toyoda, and Alex Leung. 2026. AI Identity: Standards, Gaps, and Research Directions for AI Agents. arXiv:2604.23280 [cs.CR] arXiv preprint. https://arxiv.org/abs/2604.23280.

[25] OWASP. 2025. OWASP Top 10 for Large Language Model Applications. https://owasp.org/www-project-top-10-for-large-language-model-applications/.

[26] OWASP GenAI Security Project. 2025. OWASP Top 10 Risks & Mitigations for Agentic AI. https://genai.owasp.org/.

[27] Sunil Prakash. 2026. AIP: Agent Identity Protocol for Verifiable Delegation Across MCP and A2A. arXiv:2603.24775 [cs.CR] arXiv preprint. https://arxiv.org/abs/2603.24775.

[28] Sunil Prakash. 2026. LDP: An Identity-Aware Protocol for Multi-Agent LLM Systems. arXiv:2603.08852 [cs.CR] arXiv preprint. https://arxiv.org/abs/2603.08852.

[29] Mohammadreza Rashidi. 2026. Unicode TAG-Block Concealment of Tool-Metadata Payloads in the Model Context Protocol: An Approval-View Fidelity Gap Across Three Independent Server Implementations. arXiv:2607.05744 [cs.CR] arXiv preprint. https://arxiv.org/abs/2607.05744.

[30] J. Richer, M. Jones, J. Bradley, M. Machulak, and P. Hunt. 2015. OAuth 2.0 Dynamic Client Registration Protocol. RFC 7591, IETF. https://datatracker.ietf.org/doc/html/rfc7591.

[31] Mehrdad Rostamzadeh, Sidhant Narula, Nahom Birhan, Mohammad Ghasemigol, and Daniel Takabi. 2026. MCP-DPT: A Defense-Placement Taxonomy and Coverage Analysis for Model Context Protocol Security. arXiv:2604.07551 [cs.CR] arXiv preprint. https://arxiv.org/abs/2604.07551.

[32] Christian Schroeder de Witt, Klaudia Krawiecka, Igor Krawczuk, et al. 2025. Open Challenges in Multi-Agent Security: Towards Secure Systems of Interacting AI Agents. arXiv:2505.02077 [cs.MA] arXiv preprint (24 authors). https://arxiv.org/abs/2505.02077.

[33] Riddhi Mohan Sharma. 2026. Ethical Hyper-Velocity (EHV): A Hardware-Rooted Zero-Trust Runtime Enforcement Architecture for Agentic AI Systems. arXiv:2605.17909 [cs.CR] arXiv preprint; integrates SPIFFE/SPIRE. https://arxiv.org/abs/2605.17909.

[34] Y. Sheffer, D. Hardt, and M. Jones. 2020. JSON Web Token Best Current Practices. RFC 8725, IETF. https://datatracker.ietf.org/doc/html/rfc8725.

[35] Yi Ting Shen, Kentaroh Toyoda, and Alex Leung. 2026. MCP-38: A Comprehensive Threat Taxonomy for Model Context Protocol Systems (v1.0). arXiv:2603.18063 [cs.CR] arXiv preprint. https://arxiv.org/abs/2603.18063.

[36] Tianneng Shi, Jingxuan He, Zhun Wang, Hongwei Li, Linyu Wu, Wenbo Guo, and Dawn Song. 2025. Progent: Securing AI Agents with Privilege Control. arXiv:2504.11703 [cs.CR] arXiv preprint; widely cited. https://arxiv.org/abs/2504.11703.

[37] SPIFFE Project. 2024. SPIFFE: Secure Production Identity Framework for Everyone. https://spiffe.io/.

[38] Georgios Syros, Anshuman Suri, Jacob Ginesin, Cristina Nita-Rotaru, and Alina Oprea. 2025. SAGA: A Security Architecture for Governing AI Agentic Systems. arXiv:2504.21034 [cs.CR] arXiv preprint. https://arxiv.org/abs/2504.21034.

[39] Yingzhe Tong, Leyu Dai, and Songhui Guo. 2026. AID-Guard: Stateful Authorization for Delegated Agent Effects. arXiv:2608.21159 [cs.CR] arXiv preprint. https://arxiv.org/abs/2608.21159.

[40] Uchi Uchibeke. 2026. Before the Tool Call: Deterministic Pre-Action Authorization for Autonomous AI Agents. arXiv:2603.20953 [cs.CR] arXiv preprint. https://arxiv.org/abs/2603.20953.

[41] Zhiqiang Wang, Yichao Gao, Yanting Wang, Suyuan Liu, Haifeng Sun, Haoran Cheng, Guanquan Shi, Haohua Du, and Xiangyang Li. 2025. MCP-Tox: A Benchmark for Tool Poisoning Attack on Real-World MCP Servers. arXiv:2508.14925 [cs.CR] arXiv preprint. https://arxiv.org/abs/2508.14925.

[42] Yuhao Wu, Franziska Roesner, Tadayoshi Kohno, Ning Zhang, and Umar Iqbal. 2025. IsolateGPT: An Execution Isolation Architecture for LLM-Based Agentic Systems. In *Proceedings of the Network and Distributed System Security (NDSS) Symposium*. Also arXiv:2403.04960. https://arxiv.org/abs/2403.04960.

[43] Rui Yang, Junjie Xu, Zhengyu Liu, Neil Fendley, Yang Hong, Ziyang Li, and Yinzhi Cao. 2026. SoK: When Safe Agents Fail Together: The Security of Multi-Agent LLM Systems. arXiv:2609.00595 [cs.CR] arXiv preprint. https://arxiv.org/abs/2609.00595.

[44] Hengkai Ye, Zhechang Zhang, Jinyuan Jia, and Hong Hu. 2026. TRUSTDESC: Preventing Tool Poisoning in LLM Applications via Trusted Description Generation. arXiv:2604.07536 [cs.CR] arXiv preprint. https://arxiv.org/abs/2604.07536.

[45] Xiaohang Yu, Hejia Geng, Xinmeng Zeng, and William Knottenbelt. 2026. SUDP: Secret-Use Delegation Protocol for Agentic Systems. arXiv:2604.24920 [cs.CR] arXiv preprint. https://arxiv.org/abs/2604.24920.

[46] Zehua Zhang, Jie Hu, Pratham Hegde, Aditya Maheshbhai Gabani, Souradip Nath, Yibo Liu, Siyu Liu, Hongkai Chen, Hulin Wang, Zhuoer Lyu, Chang Zhu, Divij Handa, Yan Shoshitaishvili, Tiffany Bao, Ruoyu Wang, and Adam Doupé. 2026. No-Box Vulnerability Analysis: Description-only Detection of Indirect Prompt Injection Vulnerabilities in MCP Servers. arXiv:2609.10854 [cs.CR] arXiv preprint. https://arxiv.org/abs/2609.10854.